To-do Answer the following questions in detail. Your answer should be well structured and clearly explained. Feel free to cite references where applicable.

1. Use aggregate analysis to determine the amortized cost per operation for a sequence of n operations on a data structure in which the $i\ th$ operation costs i if i is an exact power of 2, and 1 otherwise.

1) The most expensive operations is when i is a power of 2 so...

1, 2, 4, 8, 16, ... so

$\hookrightarrow 1+2+4+8+\ldots < 2n$

All other operations only cost 1, so all operations is

$2n + n = 3n$

Average Amortized cost is: $\frac{3n}{n} = O(n)$

(constant time)

2. You perform a sequence of PUSH and POP operations on a stack whose size never exceeds k. After every k operations, a copy of the entire stack is made automatically, for backup purposes. Show that the cost of n stack operations, including copying the stack, is $O(n)$ by assigning suitable amortized costs to the various stack operations. [Use the accounting method]

2) PUSH + POP cost 1, after every k operations, stack is copied = costs K

$\hookrightarrow$ So, cost 2 insted of 1

So over n operations, total amortized cost is...

$2n = O(n)$

3. Consider an ordinary binary min-heap data structure supporting the instructions INSERT and EXTRACT-MIN that, when there are n items in the heap, implements each operation in $O(log\ n)$ worst-case time. Give a potential function ϕ such that the amortized cost of i NSERT is $O(log\ n)$ and the amortized cost of EXTRACT-MIN is $O(1)$, and show that your potential function yields these amortized time bounds. Note that in the analysis, n is the number of items currently in the heap, and you do not know a bound on the maximum number of items that can ever be stored in the heap.

Used this for help with heaps
https://www.cs.cmu.edu/afs/cs/academic/class/15750-s19/OldScribeNotes/lecture2.pdf

3) BOTH insert + Extract-min take O(logn) time in worst case
Potential method $\Phi(H) = c\log n$
INSERT → real cost is O(logn), potential only changes slightly, so amortized is O(logn)
EXTRACT-MIN → real cost is O(logn) but when an element is removed, potential goes down. The drop makes the work cheaper, amortized cost goes down to O(1).

4. In a priority queue implemented using a binary heap, some operations (like insertions and deletions) might take longer when the heap needs to be restructured. Explain, in words, how amortized analysis is used to show that the average cost per operation (insert, delete) over a sequence of operations remains efficient, despite occasional restructuring of the heap.

Used this for help with heaps
https://www.cs.cmu.edu/afs/cs/academic/class/15750-s19/OldScribeNotes/lecture2.pdf

4) Heap operations may take longer because elements need to be rearranged either upwards or downwards within the heap. Most of these operations are relatively inexpensive. In amortized analysis, the rare instances of costly restructuring are averaged out, allowing the cost per operation to remain efficient.

5. Show the amortized analysis of Binary Counter implantation from our class slides using aggregate, accounting, and potential methods.

5) Aggregate method → when binary counter is incremented many times, each bit only flips occasionally
Over n increments → num of bit flips is 2n atmost
Average cost is O(1)

Accounting Method → Each increment will be charged 2 units of cost, 1 unit will pay for the bit flip now, other will be saved for later. Keeps cost at O(1)

Potential Method → Potential → number of 1s. When a lot of bits flip from 1 to 0, potential goes ↓ and helps w/ extra work later on. Cost is O(1)

6. A splay tree is a self-adjusting binary search tree in which each operation (insert, delete, search) involves a process called "splaying," which moves the accessed node to the root. Describe how amortized analysis helps demonstrate that the amortized time complexity of a series of operations on a splay tree can be bounded by $O(logn)$, even though some individual operations may take longer. Show the proof of this for the ZagZig rotation.

6) When a node is accessed in a splay tree, rotations are done to move a node towards the root. Zig-Zag is an example of this. Even if it takes extra time these rotations make the tree more balanced. Using the potential method, the potential ↓ after a rotation. This decrease helps pay for the cost, but the operations do take longer, the Amortized Analysis is still O(log n)